

CoopInfer: 边端协同推理调度原型

面向 DAG 切分、约束求解与时序可视化的桌面验证工具

CoopInfer 项目

2026 年 6 月 25 日

- 具身智能系统常由感知、融合、控制等算子组成，并形成 DAG。
- 部分算子必须固定在机器人端侧设备，部分算子可以卸载到边侧主机。
- 真实调度不仅是图切分：
 - 跨设备边会产生网络传输代价；
 - 端侧和边侧都有有限执行队列；
 - 端到端时延上限会直接排除不可行方案。
- CoopInfer 用于快速编辑、求解、验证并可视化这类调度方案。

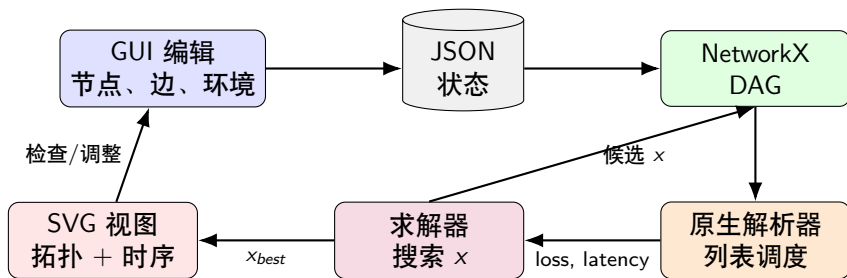
输入

- DAG 节点及端侧/边侧执行耗时。
- 边数据量。
- 带宽、固定时延和优化权重。
- 串行网络模型以及可选传输批处理。
- 可选平均端到端时延上限和最大单帧时延上限。

输出

- 节点放置决策: $x_i = 0$ 表示端侧, $x_i = 1$ 表示边侧。
- 端到端时延和资源利用率。
- SVG 拓扑图和 profiler 风格时序图。

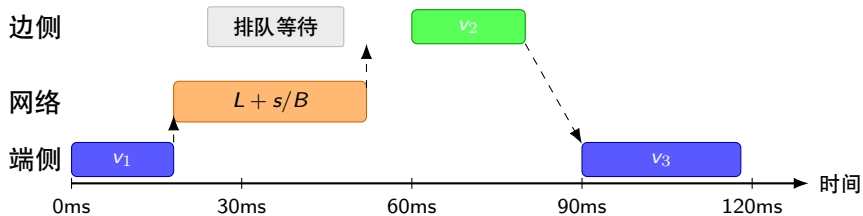
原型闭环



原型把编辑、求解、物理推演和可视化放在一个可复现实验闭环中。

- 内存中使用 `networkx.DiGraph` 存储拓扑。
- 节点属性：
 - `id`, `name`, `c_dev`, `c_host`, `fixed_dev`, `x`
 - 可选输入频率字段: `source_period_ms`, `source_phase_ms`
- 边属性：
 - `source`, `target`, `size`, 单位为 MB。
- 环境参数：
 - `bandwidth`, `latency`, `weight_avg_latency`, `weight_max_latency`, `weight_device_utilization`
 - `latency_limit`, `max_frame_latency_limit`, `batch_transfers`, `pipeline_unroll`
- JSON 导入/导出保证实验可复现、可共享。

考虑队列的执行过程



节点必须同时满足“数据已就绪”和“目标设备队列空闲”两个条件才能开始执行。

内层解析器步骤 1: 展开任务 DAG

对一个候选放置方案 x , C++ 核心会先把展开后的流水线构造成任务 DAG。

- Source 任务: 按 $phase_i + f \cdot period_i$ 释放的零耗时输入事件。
- Compute 任务: 每个非输入节点、每一帧各生成一个计算任务。
- 跨设备边: 显式 Network 任务, 耗时为 $L + s/B$ 。
- 批处理跨设备出边: 一个 Network 任务, 耗时为 $L + \sum_k s_k/B$ 。

依赖类型包括:

- 同设备数据边: 源计算/输入任务 $\rightarrow$ 目标计算任务。
- 跨设备数据边: 源任务 $\rightarrow$ Network 任务 $\rightarrow$ 目标任务。
- 同一非输入算子的相邻帧: 添加从第 f 帧到第 $f+1$ 帧的 FIFO 边。

内层解析器步骤 2：就绪列表事件循环

仿真器维护剩余依赖数量、最早数据就绪时间，以及三类串行资源的空闲时间：

$$time_dev_ready, \quad time_host_ready, \quad time_network_ready$$

就绪任务的开始时刻为：

$$S_t = \begin{cases} ready_t, & resource(t) = source \\ \max(ready_t, time_dev_ready), & resource(t) = device \\ \max(ready_t, time_host_ready), & resource(t) = host \\ \max(ready_t, time_network_ready), & resource(t) = network \end{cases}$$

主循环：

- ① 把剩余依赖数为 0 的任务加入就绪集合。
- ② 选择可行开始时间最早的就绪任务。
- ③ 若开始时间相同，则用当前启发式规则打破平局。
- ④ 记录开始/结束时间，更新资源队列，并释放后继任务。

内层解析器步骤 3：多规则集成

每个候选方案只构建一次任务 DAG 和向上 rank:

$$rank(t) = duration(t) + \max_{u \in succ(t)} rank(u)$$

解析器会仿真七种确定性规则:

- **CriticalPath**: 优先级为 rank 加老帧加成

$$rank(t) + 10^6 \max(0, unroll - 1 - frame(t)).$$

- **DeviceFirst**: Source、端侧、Network、边侧依次优先; 同类用耗时和 rank 打破平局。
- **HostFirst**: Source、边侧、Network、端侧依次优先。
- **NetworkFirst**: Source、Network、端侧、边侧依次优先。
- **OutputFirst**: 输出计算任务优先, 然后按老帧加成和 rank 排序。
- **Throughput**: 年轻帧加成和 rank, 用于测试稳态流水调度。
- **FifoReady**: 在相同可行开始时间下按任务创建顺序稳定排序。

先选最早可行开始时间保证资源不无谓空转; 规则只负责同一开始时刻的优先级选择。

内层解析器步骤 4：尾帧时延重排

解析器会降低人为拉长的尾帧 E2E，但不添加跨帧全局门控：

- **延迟受阻传输**：若跨设备目标还在等待其他前驱，则把传输推迟到这些前驱完成后再启动。
- **右移扫描**：列表调度完成后，把有余量的网络任务和 join 支路计算向后移动，但必须保持后继、资源顺序、释放时间和输出完成时刻不变。

这样可以避免“过早开始但无法推进输出”的任务被计为该帧第一个非输入事件，同时保留真实的跨帧流水并行。

每个候选方案共评估：

$$2 \times 2 \times 7 = 28$$

套确定性列表调度：两种传输时序、是否右移重排，以及七种就绪列表规则。

内层解析器步骤 5：指标提取与胜者选择

时延不包含输入源事件。对每一帧：

$$frame_start = \min(\text{非输入计算开始、传输开始})$$

$$frame_finish = \max(\text{非输入输出节点完成})$$

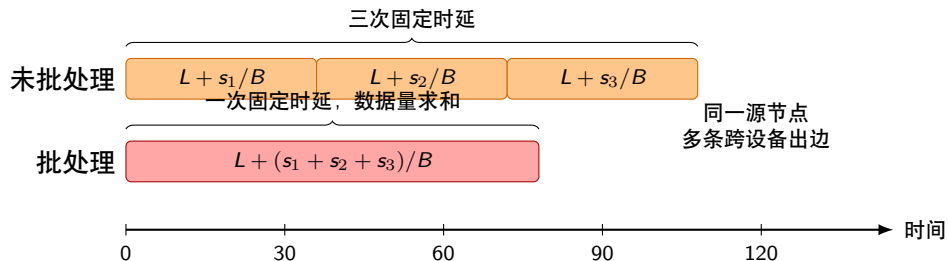
$$\tau_{frame} = frame_finish - frame_start$$

输出指标：

$$\tau_{max} = \max_f \tau_{frame}(f), \quad \tau_{avg} = \frac{pipeline_finish - pipeline_start}{pipeline_unroll}$$

最终保留 Loss 最低的调度；若 Loss 相同，依次选择最大单帧时延更低、平均时延更低、启动间隔更低、端侧利用率更高的方案。

网络批处理示意



批处理减少重复固定时延，但不会让网络传输发生重叠。

网络模型：串行传输与批处理

- 所有跨设备传输共享一条网络工作队列：

$$time_network_ready$$

- 传输必须等待源节点完成，并且等待网络队列空闲，因此不同传输不会重叠。
- 未开启批处理时，每条跨设备边独立计费：

$$T_{edge} = L + \frac{s}{B}$$

- 开启 `batch_transfers=true` 后，同一个源节点的多条跨设备出边会作为一次网络事务发送：

$$T_{batch} = L + \frac{\sum_k s_k}{B}$$

- 这样既保持网络非重叠，又避免连续出边重复支付固定时延。

优化目标与可行性约束

求解器最小化加权目标：

$$J = w_{avg}L_{avg} + w_{max}L_{max} + w_{util}L_{util}$$

其中

$$L_{avg} = \frac{\tau}{\max(scale_{avg}, 1)}, \quad L_{max} = \frac{\tau_{frame}^{max}}{\max(scale_{max}, 1)}, \quad L_{util} = 1 - \frac{device_active_time}{pipeline_work_span}$$

时延归一化尺度来自全端侧/全边侧基线。

时延上限

正数上限是硬约束：

$$\tau \leq latency_limit, \quad \tau_{frame}^{max} \leq max_frame_latency_limit$$

取值为 0 表示关闭对应约束；超过任一启用上限的候选方案会在目标函数比较前被直接拒绝。

求解模式

模式	适用场景	行为
Auto Enumerate	默认模式 小规模 DAG	自由节点 ≤ 12 时枚举，否则随机搜索 将 assignment mask 按线程切分，返回精确全局最优可行解
Random Search Simulated Annealing	较大 DAG 跳出局部最优	多线程独立扰动链，保留最佳可行候选 多线程独立退火链，支持配置初始/最终温度

`solver_threads=0` 表示使用硬件线程数。所有模式都会强制固定端侧节点，并执行平均端到端和最大单帧时延上限过滤。

GUI 使用流程

- ① 编辑节点、名称、计算耗时和固定端侧标记。
- ② 编辑边和传输数据量。
- ③ 配置带宽、固定时延、网络批处理、优化权重、求解模式、线程数、迭代次数、退火温度和时延上限。
- ④ 点击 **Solve**。
- ⑤ 查看时延、资源利用率、拓扑放置和 profiler 风格时序图。

系统会校验重名节点、非法边端点以及非 DAG 拓扑。

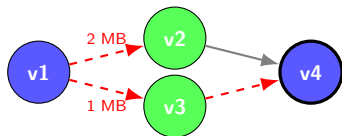
拓扑 SVG

- 配置 DAG 在求解前展示数据量以及本地/跨设备边样式。
- 求解后的展开流水线使用高对比填充色表示帧编号。
- 蓝色轮廓表示端侧，绿色轮廓表示边侧。
- 加粗或虚线源节点样式表示固定端侧和输入源事件。
- FIFO、传输、本地依赖和释放线都跟随源帧的同一颜色。

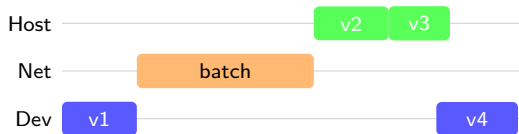
时序 SVG

- Input、Host、Transfer、Device 四条泳道。
- 算子条由开始/完成时间决定，并使用与拓扑图一致的帧填充色。
- 传输条来自解析器的真实传输记录，可展示串行传输和批处理传输。
- 输入释放点、周期跨度和垂直释放/依赖线都使用对应帧颜色。
- 使用浏览器原生滚动和缩放控件。

拓扑图与时序图读法

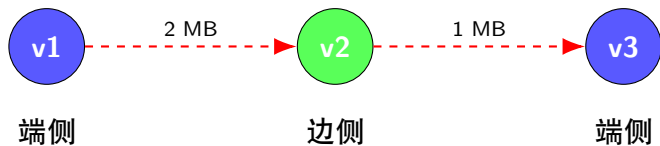


拓扑图：颜色表示放置位置，边样式表示通信关系



时序图：条形位置来自解析器计算出的开始/完成时间

示例 DAG



解析器输出的是考虑设备队列的真实调度，而不是理想无限并行关键路径。

当前技术栈

- Python 3.9+。
- PyQt6 负责桌面控件和布局。
- PyQt6-WebEngine 负责 SVG 拓扑图和时序图渲染。
- NetworkX 负责 DAG 存储、拓扑排序和环路检测。
- C++/pybind11 原生核心负责调度评估、目标函数计算和求解搜索。
- Pytest 覆盖解析器、JSON IO、求解模式和时延上限可行性。
- 启动命令: `python -m coopinfer` 或 `python -m coopinfer.app`。

- 增加求解诊断：被拒绝候选、约束裕量、收敛历史。
- 支持拓扑图和时序图导出为 SVG/PNG 报告。
- 增加带宽/时延参数扫描的批量评估。
- 展示被选中的调度规则、队列裕量和就绪任务诊断信息。
- 在时序图中显式区分队列等待、数据就绪和传输阶段。
- 探索更多网络重排策略和传输优先级规则。

Q&A